1 . Three Layer Neural Network

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()


X_train = X_train.reshape(-1, 784) / 255.0
X_test = X_test.reshape(-1, 784) / 255.0


X_train = X_train[:10000]
y_train = y_train[:10000]


def one_hot(y, num_classes=10):
    result = np.zeros((y.shape[0], num_classes))
    result[np.arange(y.shape[0]), y] = 1
    return result

y_train_onehot = one_hot(y_train)


np.random.seed(42)

W1 = np.random.randn(784, 128) * np.sqrt(2 / 784)
b1 = np.zeros((1, 128))

W2 = np.random.randn(128, 10) * np.sqrt(2 / 128)
b2 = np.zeros((1, 10))


def relu(x):
    return np.maximum(0, x)

def relu_derivative(x):
    return (x > 0).astype(float)

def softmax(x):
    x = x - np.max(x, axis=1, keepdims=True)
    exp_x = np.exp(x)
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

def cross_entropy(y_true, y_pred):
```

```
    y_pred = np.clip(y_pred, 1e-9, 1 - 1e-9)
    return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))


learning_rate = 0.1
epochs = 20
batch_size = 64


for epoch in range(epochs):


    indices = np.random.permutation(len(X_train))
    X_train = X_train[indices]
    y_train_onehot = y_train_onehot[indices]

    total_loss = 0

    for i in range(0, len(X_train), batch_size):

        X_batch = X_train[i:i + batch_size]
        y_batch = y_train_onehot[i:i + batch_size]


        Z1 = np.dot(X_batch, W1) + b1
        A1 = relu(Z1)

        Z2 = np.dot(A1, W2) + b2
        A2 = softmax(Z2)


        loss = cross_entropy(y_batch, A2)
        total_loss += loss


        dZ2 = A2 - y_batch
        dW2 = np.dot(A1.T, dZ2) / len(X_batch)
        db2 = np.sum(dZ2, axis=0, keepdims=True) / len(X_batch)

        dA1 = np.dot(dZ2, W2.T)
        dZ1 = dA1 * relu_derivative(Z1)

        dW1 = np.dot(X_batch.T, dZ1) / len(X_batch)
        db1 = np.sum(dZ1, axis=0, keepdims=True) / len(X_batch)


        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2
```

```
        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1


    Z1 = np.dot(X_train, W1) + b1
    A1 = relu(Z1)

    Z2 = np.dot(A1, W2) + b2
    A2 = softmax(Z2)

    predictions = np.argmax(A2, axis=1)
    actual = np.argmax(y_train_onehot, axis=1)

    accuracy = np.mean(predictions == actual)

    print(
        f"Epoch {epoch + 1}/{epochs} "
        f"- Loss: {total_loss / (len(X_train) // batch_size):.4f} "
        f"- Accuracy: {accuracy * 100:.2f}%"
    )


Z1 = np.dot(X_test, W1) + b1
A1 = relu(Z1)

Z2 = np.dot(A1, W2) + b2
A2 = softmax(Z2)

test_predictions = np.argmax(A2, axis=1)

test_accuracy = np.mean(test_predictions == y_test)

print("\nTest Accuracy:", test_accuracy * 100, "%")


plt.figure(figsize=(12, 6))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap="gray")
    plt.title(f"Actual: {y_test[i]}\nPredicted: {test_predictions[
    plt.axis("off")

plt.tight_layout()
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/20 - Loss: 0.7223 - Accuracy: 87.39%
Epoch 2/20 - Loss: 0.3502 - Accuracy: 91.63%
Epoch 3/20 - Loss: 0.2915 - Accuracy: 92.75%
Epoch 4/20 - Loss: 0.2557 - Accuracy: 93.39%
Epoch 5/20 - Loss: 0.2278 - Accuracy: 92.08%
Epoch 6/20 - Loss: 0.2059 - Accuracy: 94.71%
Epoch 7/20 - Loss: 0.1850 - Accuracy: 95.54%
Epoch 8/20 - Loss: 0.1698 - Accuracy: 95.75%
Epoch 9/20 - Loss: 0.1531 - Accuracy: 95.31%
Epoch 10/20 - Loss: 0.1427 - Accuracy: 95.42%
Epoch 11/20 - Loss: 0.1294 - Accuracy: 96.94%
Epoch 12/20 - Loss: 0.1198 - Accuracy: 96.82%
Epoch 13/20 - Loss: 0.1113 - Accuracy: 96.42%
Epoch 14/20 - Loss: 0.1011 - Accuracy: 97.77%
Epoch 15/20 - Loss: 0.0938 - Accuracy: 98.03%
Epoch 16/20 - Loss: 0.0862 - Accuracy: 98.18%
Epoch 17/20 - Loss: 0.0802 - Accuracy: 98.51%
Epoch 18/20 - Loss: 0.0754 - Accuracy: 98.53%
Epoch 19/20 - Loss: 0.0696 - Accuracy: 98.73%
Epoch 20/20 - Loss: 0.0647 - Accuracy: 98.62%

Test Accuracy: 94.46 %
```

